Modern Metrics (MM): Functional Size Estimator for AI and BDA Software Applications

B. Senthilkumaran, John T. Mesia Dhas, N. Kurinjivendhan, and M. Usha

Abstract Analysing, scheduling, developing, testing, and maintaining are the main activities of software project management (SPM) under software engineering. Software size is the major factor determining cost, time, and effort for developing software applications. The successful completion of all the activities of SPM requires an accurate determination of the cost, time, and effort required for developing the application. That is, the real size of the software application is the major determinant of all the SPM activities. The existing traditional application-oriented and single-dimensional software sizing techniques will not give the actual size of modern software applications like artificial intelligence (AI) and big data analytics (BDA). The imperfect estimation leads to wrong predictions about the cost, time, and effort required for software application development. The wrong estimates will affect the entire SPM activities, resulting in a delay in delivery, a loss in predicted cost, an imbalanced effort level, and the dismissal of the software development process. The major software industries are facing the same problem with wrong size estimates. The updated and dynamic new size estimation technique is highly essential for this current SPM environment. The novel approach of modern metrics (MM) addresses all the technical challenges facing software engineering for the software size estimation process. This article analyses the major issues of the software size estimation process, major functional and non-functional units associated with software size estimation, the architecture of MM, and an algorithm for software size estimation using MM.

B. Senthilkumaran (✉) · J. T. Mesia Dhas
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology (Deemed to be University), Chennai 600 062, India
e-mail: skumaran.gac16@gmail.com

J. T. Mesia Dhas
e-mail: jtmdhasres@gmail.com

N. Kurinjivendhan
Bishop Heber College, Trichy 620 017, India
e-mail: vendhancs1489@gmail.com

M. Usha
Sri Amaraavathi College of Arts and Science (Affiliated to Bharathidasan University), Karur, India
e-mail: usharavichandran@amaraavathi.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2025

H. Zen et al. (eds.), *Soft Computing and Signal Processing*, Lecture Notes in Networks and Systems 1200, https://doi.org/10.1007/978-981-97-9926-8_25

Keywords Software project management · Software size estimation · Modern metrics · Software complexity factors · Software effort · Software schedule · Modern metrics size · Software engineering

1 Introduction

Software engineering is an engineering procedure for creating software application projects in an international standard manner, giving guidelines for completing the project on time, testing and debugging in user as well as developer-centric approaches, and providing the complete procedure for maintaining the developed software. Modern software systems use about one programming language, application software, utility tool, architecture, platform (operating system), type of user, and databases. So, the development of a modern software system is highly complex. Modern applications are founded on software as well as hardware. The systems (both hardware and software) are generating a huge amount of data. That data must be processed in parallel with the next process [1].

The modern industry is developing applications in artificial intelligence (AI) and the Internet of things (IoT). The applications are processing a huge amount of data (GB to PB) to generate a simple result, which uses different types of datasets (structured, unstructured, and semi-structured), different sets of database management systems (DBMS), software architectural models, etc. Emerging automation technologies like IoT use sensors, actuators, clouds for storage, big data analytics (BDA) tools, and highly efficient programming languages. AI techniques such as machine learning (ML) and deep learning (DL) analyse historical data. To develop such a complex application requires about one programming language, utility apps, architectural models, DBMS tools (for handling structured, semi-structured, and unstructured data), domain experts (for handling problems in different domains of software), storage (static and dynamic), parallel processing processors, network topologies, distributed servers, virtual processing, etc.

Developing and delivering this complex software product on schedule is a very difficult task. A software engineering study says that less than 30% of the software is delivered on time. The latency in delivery leads to economic losses for both the customer and the developer. The trust of the developer is challenged. One of the important reasons for this delayed delivery is a lack of planning. In the planning process, software size estimation plays a crucial role. The estimation of effort, cost, and time for developing the software is determined by the software size factor [2, 3].

Software is an intangible, logical product. So, the estimation of size is highly risky. The software industry is using many traditional size estimation techniques like lines of code (LoC), function point (FP), feature point, Delphi, expert judgement, pattern matching, etc. In terms of software sizing techniques, the FP method is the most reliable, programming language-independent, user-centric, and understandable to beginners. The FP method deals with the size of the software at the analysis front itself. So, the estimation of effort, cost, and time is possible at the initial phase itself.

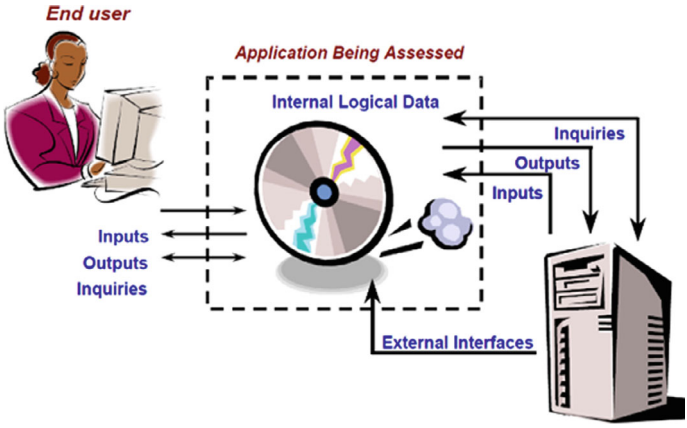

Fig. 1 FP architecture

Around 90% of the software industries in the world are following the FP method for measuring the size of the software [4, 5].

In the 1970s, Alan Albrecht introduced the FP method for determining the size of the software [5–10]. The key features of the FP method are as follows:

- Independent of technology, platform, or language.
- Excellent as an initial step for estimating development costs and timelines.
- It provides the ability to normalise metrics for consistent analysis.
- Fully documentable and repeatable.
- Aids in requirement management; functional size is traceable throughout the life cycle.
- Provides a quantitative basis for earned value management.

The architecture of the FP method is presented in Fig. 1.

2 Literature Review

According to the research based on Capers Jones [1–5], the lines of code (LoC) or counting of lines in a computer software application is the size of the software application. The programming languages like COBOL and FORTRAN increased the logical ability of the software applications. The weight of code used in one application may be varied in another application. So, the LoC method does not opt for finding the size of all the software applications. Modern software applications vary with distinct domains, scientific and static calculations, embedding with other technologies, multipurpose, and versatility. Finding the size is highly complex in modern days.

An expert or a panel of experts were responsible for finding the size of the software application based on their assumptions and decisions [2]. To count the operands and operators present in the software project is another way of identifying the size [2]. The effort level of the software application development process is based on the programming languages used for the software application development activity [1]. To count inputs, outputs, inquiries, logical files, and interfaces, another method is known as function points. It was developed in 1965 by Allan Albrecht [1–5].

Nasir and Ahmad [6] argue that the size estimation at the earlier stage in SPM helps reduce the time, effort, and cost required for software applications. In modern software systems, a single method is not sufficient for estimating the real size of the application.

Kaya et al. [8] found that the effort, time, and cost of the software development process are based on the size of the software application.

According to Daniel V. Feren et al. [10], an application-independent software size estimation technique is required for the modern software industry.

Boehm [10, 11] says that estimating the actual software size is the biggest problem in the current software industry.

Zia et al. [12] found that software size plays a major role in the planning, analysis, risks, and budgeting of SPM.

According to Rabbi et al. [13], a new effective cost estimation technique is required for this mature software industry. Finding the size with the existing technique is not accurate.

Objectives

1. To identify the missing FPs in modern AI and big data analytics (BDA) software systems.
2. To identify all complexity adjustment factors (CAF) affecting the SDLC
3. To derive a novel methodology for identifying software size.
4. To derive a formula to identify the effort level of the software.
5. To derive a formula to identify the essential time for developing software.
6. To derive a formula to calculate the cost required to develop the software product.

Motivation

The software industry is a highly dynamic and variable industry that depends on time and situations. The size estimation techniques do not match the current industrial flow. So many defects exist in the assessments based on the present techniques, which is affecting the delivery of the software product. To overcome the current scenario, the industry requires an updated estimation technique for identifying the size of the software.

3 Modern Metrics

Modern metrics (MM) is a proposed novel methodology for identifying the size of software products like AI and BDA applications. It follows Alan Albrecht's FPA method with new and updated function points and complexity adjustment factors (CAF). This method is language-, platform-, and technology-independent. The resultant value of this method is an important factor for estimating the time, effort, and total cost of the software product. The estimation process is fully documentable and repeatable. This method is capable of identifying the size of the software at any stage of the SDLC. The estimation process gives effective outcomes that are easily understandable by any kind of user.

MM Architecture

The MM is comprised of FPs and CAFs. The MM architecture is illustrated in Fig. 2

II: Internal Inputs, IO: Internal Operations, ILF: Internal Logical Files, EIF: External Interface Files, CAF: Complexity Adjustment Factors, SDLC: Software Development Life Cycle, MMSize: Modern Metrics Size, UMMFP: Unadjusted MM Function Points, and MMCAF: MM Complexity Adjustment Factor.

The architecture explains all of a software's internal, external, and fusion functional units; both internal and external complexity factors influence the SDLC. The size of the software application is the multiplied outcome of UMMFP and MMCAF.

A. *Functional Units of MM*

The functional units (FU) of software are the foundation for estimating its size. The values associated with each functional unit are known as its factors. The factors affecting the FU of modern software systems are tough to find and classify. The important factors of FUs are identified: internal inputs (II), internal operations (IO), internal

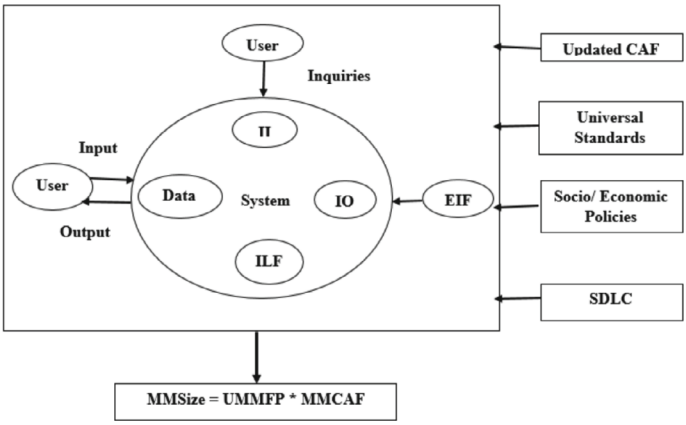

Fig. 2 MM architecture

logical files (ILF), external inputs (EI), external outputs (EO), external inquiries (EQ), external interface files (EIF), and data and text (DT). The MM is using eight different types of functional values associated with software projects. Each functional unit has its own metrics and metric values. The metrics are otherwise known as effort modifiers. Based on the user's efforts, the standards result in individual metrics for the functional unit.

The researchers analysed seven AI and BDA projects of Palm Services; the results in Table 1 above are identified. It says that an average of 0.98 is the effort required for developing an II functional unit from the analysis phase to the implementation phase. The average effort of II varies from 0.57 to 1.63. The study says that when the number of IIs increases in each function of a project, it also increases the effort level of the project. The collective effort is less than the individual effort.

The analysis results of seven projects for internal operations are tabulated in Table 2. The average effort required for each IO functional unit is 1.1. The effort levels are high in scientific and analytical software and low in application-based software. The average effort in collective and individual efforts is mostly the same.

The results in Tables 3 and 4 show the efforts required to complete external input. The values vary from software to software, depending on the complexity and technological novelty. The collective effort is less because some of the same EIs are also used by other functions. It reduces the effort required in the earlier phases of SDLC.

The effort required to finish the functional unit EO is listed in Table 5. The collective effort is slightly less than the effort required to finish an individual functional unit. The complexity level of the software application is the major reason for this minor difference.

Table 1 Internal input effort estimation

S. No.	Project title	MM functional unit					
		Internal input					
		no. of II functional units	Effort	No. of functions	Average II per function	Average effort per function	Average effort per FU
1	Project 1	40	24	42	0.95	0.57	0.60
2	Project 2	92	58	86	1.07	0.67	0.63
3	Project 3	128	117	72	1.78	1.63	0.91
4	Project 4	71	63	43	1.65	1.47	0.89
5	Project 5	18	24	39	0.46	0.62	1.33
6	Project 6	112	103	83	1.35	1.24	0.92
7	Project 7	59	43	64	0.92	0.67	0.73
Total		520	432	429			
Average						0.67	0.83

Table 2 Internal operations effort estimation

S. No.	Project title	MM functional unit					
		Internal operations					
		No. of IO functional units	Effort	No. of functions	Average IO per function	Average effort per function	Average effort per FU
1	Project 1	40	32	42	0.95	0.76	0.80
2	Project 2	68	44	86	0.79	0.51	0.65
3	Project 3	167	178	72	2.32	2.47	1.07
4	Project 4	42	31	43	0.98	0.72	0.74
5	Project 5	42	36	39	1.08	0.92	0.86
6	Project 6	174	143	83	2.10	1.72	0.82
7	Project 7	49	36	64	0.77	0.56	0.73
Total		582	500	429			
Average						0.86	0.86

Table 3 Internal logical files effort estimation

S. No.	Project title	MM functional unit					
		Internal logical files					
		No. of ILF functional units	Effort	No. of functions	Average ILF per function	Average effort per function	Average effort per FU
1	Project 1	42	40	42	1.00	0.95	0.95
2	Project 2	96	69	86	1.12	0.80	0.72
3	Project 3	64	82	72	0.89	1.14	1.28
4	Project 4	46	41	43	1.07	0.95	0.89
5	Project 5	39	26	39	1.00	0.67	0.67
6	Project 6	86	94	83	1.04	1.13	1.09
7	Project 7	67	49	64	1.05	0.77	0.73
Total		440	401	429			
Average						1.07	0.91

Table 6 lists the effort required to do the external inquiries from the beginning to the end of the SDLC cycle. The collective effort is slightly higher than the individual effort. Modern software systems use more EQ compared with the previous decade.

Table 7 lists the effort required for developing EIF functional units in the software system. Modern software uses many in-built and online external resources for the betterment of software quality. In this research, the identification is that the collective effort is a little higher than the individual effort. The collective functional units

Table 4 External input effort estimation

S. No.	Project title	MM functional unit					
		External input					
		No. of EI functional units	Effort	No. of functions	Average EI per function	Average Effort per function	Average effort per FU
1	Project 1	92	71	42	2.19	1.69	0.77
2	Project 2	173	193	86	2.01	2.24	1.12
3	Project 3	192	151	72	2.67	2.10	0.79
4	Project 4	61	42	43	1.42	0.98	0.69
5	Project 5	48	31	39	1.23	0.79	0.65
6	Project 6	187	192	83	2.25	2.31	1.03
7	Project 7	173	142	64	2.70	2.22	0.82
Total		926	822	429			
Average						0.52	0.89

Table 5 External output effort estimation

S. No.	Project title	MM Functional unit					
		External output					
		No. of EO functional units	Effort	No. of functions	Average EO per function	Average effort per function	Average effort per FU
1	Project 1	56	65	42	1.33	1.55	1.16
2	Project 2	53	61	86	0.62	0.71	1.15
3	Project 3	83	89	72	1.15	1.24	1.07
4	Project 4	31	40	43	0.72	0.93	1.29
5	Project 5	23	13	39	0.59	0.33	0.57
6	Project 6	44	76	83	0.53	0.92	1.73
7	Project 7	31	31	64	0.48	0.48	1.00
Total		321	375	429			
Average						1.14	1.17

require some modifications for their customization. It is increasing the effort level in the collective state of operations.

The effort required for the functional unit DT is listed in Table 8. Modern software systems have a high data usage rate. The effort levels are lower for online content, images, social media information, etc. However, the effort level for traditional (hand-typing) data is high (project 5 required 59 human efforts to generate 31 GB of data). The effort level for the current software application decreases as the DT value increases.

Table 6 External inquiries effort estimation

S. No.	Project title	MM functional unit					
		External inquiries					
		No. of EQ functional units	Effort	No. of functions	Average EQ per function	Average effort per function	Average effort per FU
1	Project 1	12	18	42	0.29	0.43	1.50
2	Project 2	15	24	86	0.17	0.28	1.60
3	Project 3	8	8	72	0.11	0.11	1.00
4	Project 4	6	16	43	0.14	0.37	2.67
5	Project 5	41	67	39	1.05	1.72	1.63
6	Project 6	21	38	83	0.25	0.46	1.81
7	Project 7	18	23	64	0.28	0.36	1.28
Total		121	194	429			
Average						2.21	1.60

Table 7 EIF effort estimation

S. No.	Project title	Modern Metrics (MM) Functional Unit					
		EIF					
		No. of EIF functional units	Effort	No. of functions	Average EIF per function	Average effort per function	Average effort per FU
1	Project 1	85	78	42	2.02	1.86	0.92
2	Project 2	146	117	86	1.70	1.36	0.80
3	Project 3	93	71	72	1.29	0.99	0.76
4	Project 4	89	58	43	2.07	1.35	0.65
5	Project 5	68	53	39	1.74	1.36	0.78
6	Project 6	116	82	83	1.40	0.99	0.71
7	Project 7	82	63	64	1.28	0.98	0.77
Total		679	522	429			
Average						0.82	0.77

B. *FUs with MM Metrics and Metric Values*

The metrics are systemic or a standard of measurement. In MM, the eight functional units are ordered according to their importance in a function. The metrics of the functional units are low, average, high, and very high based on the standard effort required to complete the operations of each functional unit. By using a set of inflexible standards, the size of the application, the influence of functional units in the function,

Table 8 Data and text effort estimation

S. No.	Project title	MM functional unit					
		Data & text					
		No. of DT functional units	Effort	No. of functions	Average DT per function	Average effort per function	Average effort per FU
1	Project 1	60	45	42	1.43	1.07	0.75
2	Project 2	380	274	86	4.42	3.19	0.72
3	Project 3	212	90	72	2.94	1.25	0.42
4	Project 4	68	51	43	1.58	1.19	0.75
5	Project 5	31	59	39	0.79	1.51	1.90
6	Project 6	173	139	83	2.08	1.67	0.80
7	Project 7	186	162	64	2.91	2.53	0.87
Total		1110	820	429			
Average						0.52	0.74

and the time and effort required for the advancement of a functional unit, the metrics are categorised in the following Table 9.

The industrial studies with live projects say that the metrics of FUs units are categorised. If a function has 1 to 3 EI, then the metric of EI in that function is low. If a function has 4 to 5 EI, then the metric of EI in that function is average. If a function has 6 to 8 EI, then the metrics in the function are high. If a function has more than 8 EI, then the metric of EI in that function is very high. Similarly, all the metrics are identified based on the effort required to develop that functional unit from the analysis phase to the enhancement phase. The metric values of FUs of MM are itemised in the following Table 10.

The industrial study with the live projects gives the average effort required for the functional units to be developed. If the EI metric is low, then the average effort required to complete the functional unit is 3. If the EI metric is average, then the average effort required to complete the functional unit is 4. If the EI metric is high, then the average effort required to complete the functional unit is 6. If the metric is very high, then the average effort required to complete the functional unit is 9.

Table 9 Functional units with metrics

Metrics	Functional units							
	EI	II	EO	IO	DT	EQ	ILF	EIF
Low	1–3	1–3	1–4	1–3	1–4	1–3	1–7	1–5
Average	4–5	4–5	5–6	4–5	5–6	4–5	8–14	6–9
High	6–8	6–8	7–9	6–8	7–9	6–8	15–21	10–13
Very high	> 8	> 8	> 9	> 8	> 9	> 9	> 21	> 13

Table 10 Metrics with its values

Metrics	Functional units							
	EI	II	EO	IO	DT	EQ	ILF	EIF
Low	3	3	4	3	4	3	7	5
Average	4	4	5	4	5	4	10	7
High	6	6	7	6	7	6	15	10
Very High	9	9	10	9	10	9	22	14

C. *Calculating Functional Units of Software Application using MM*

The software application must be analysed module-wise and list out the count of all the functional units present in each function of the application. The values are tabulated in Table 11 below.

The above table is helpful for tabulating and finding entire FUs in an application, all the functions referred to for each functional unit, and the total count for each FU in a category.

CAF of Software using MM

The complexity level of modern software applications is derived from the following reasons:

1. A number of technologies were incorporated into the software.
2. International standardisation and development models are associated with the software.
3. The operations of distributed and parallel processing applications.
4. The novelty and innovation techniques are complicated in the emerging system.
5. Uncertainties (failures, deadlocks, policies, etc.) associated with the software application system.

The following are the most influential factors and their values: Nil—0, Secondary—1, Moderate—2, Average—3, Important—4, and Essential—5, . All

Table 11 Calculating FUs

S. No.	Function name	Functional units							
		EI	II	EO	IO	DT	EQ	ILF	EIF
1									
2									
3									
4									
5									
Total functions used (TF)									
Total FUs (TFU)									

the complexity metrics are recognised and estimated by the above Table 11. The MMCAF is identified using the formula:

$$CF_i = 0.25 * 0.01 * MMCAF \quad (1)$$

The CFi (where i = 1 to 22) is the level of impact and involvement of CF in software applications, particularly on responses collected from the CAF table. The MMCAF does not exceed 35%.

D. *Unadjusted MM Function Points (UMMFP)*

The sum of functional values existing in software applications resulting from the number of functional values present individually at functions of the software application system is UMMFP. The procedure for UMMFP uses the following Table 12:

The product of the TF and W is UMMFP. The AFU is the proportion of TFUs and TFs. The weight or matrix value is particularly on the entire FUs existing in the AFU for each individually functional unit in the entire system. In MM, the average value of the entire system is calculated instead of each function or module. The calculation of function points for each function and module is also possible with MM.

The UMMFP is intended to be founded on the parameters in Table 6. It is the parameter for a conclusion: the size of the software.

E. *MMSize*

The MMSize, or modern metric size, is the real size of software based on MM function points (MMFP). The product of UMMFP and MMCAF is MMSize. That is,

$$MM\,Size = MMCAF * UMMFP \quad (2)$$

Table 12 Unadjusted MMFP

S. No.	Functional units	Total functions (TF)	Total functional units (TFU)	Average functional units (AFU = TFU/TF)	Metrics	Metric value (W)	UMMFP (TF * W)
1							
2							
3							
4							
5							
6							
7							
8							
Total UMMFP							

The MMSize not only considers the functional values present in the software but also considers the complexity of the software in all the possible technologies. So, the accuracy of the software is increasing compared with traditional FP methods.

F. *Advantages of MM*

The highlighted advantages of MM are as follows:

1. The MM technique is a novel technique for finding the size of the software in any phase of the SDLC and also finding the size after the completion of the software development.
2. The MM technique finds the effort required for the development of the functional units in an individual and collective manner. So, the defects present in the FP count are negligible.
3. The complexity adjustment factors and effort metrics are modified and updated based on the requirements of the modern software system.

G. *Future Developments*

The future enhancements in MM are as follows:

1. The MM technique was developed based on Indian software industry values. So, the analysis must be done using international values.
2. The modern software industry is using a greater number of functional units with a function. So, it is facing difficulties in fixing the limits of the very high metric of functional values.

4 Conclusions

The modern software industry is waiting for an effective size estimator with updated parameters for solving software management-based technical, economic, and social issues. The proposed MM analyses all the issues associated with the software from a user and developer perspective. It gives the feature of estimating the size of the software from any of its SDLC phases. The earlier estimation helps the software industry to do the development process with the derived time, cost, and effort. It increases the efficiency of delivery. The analysis-based work of the MM is progressive with industrial parameters.

References

1. Jones C (2008) Applied software measurement-global analysis of productivity and quality, 3rd edn. Tata McGraw Hill, pp 71–182
2. Stutzke RD (2005) Estimating software-intensive systems: projects, products and processes. SEI series in software engineering. Addison Wesley Edition, pp 1–786
3. Jones C (2007) Estimating software costs: bringing realism to estimating, 2nd edn. Tata McGraw Hill, pp 3–629
4. Jones C (2010) Software engineering best practices: lessons from successful projects in the top companies, edn. Tata McGraw Hill, pp 1–643
5. Futrell RT, Shafer DF, Shafer LI (2008) Quality software project management. Pearson Education, pp 1–600
6. Nasir M, Farooq Ahmad H (2006) An empirical study to investigate software estimation trend in organizations targeting CMMISM. In: Proceedings of the 5th IEEE/ACIS international conference on computer and information science and 1st IEEE/ACIS international workshop on component-based software engineering, software architecture and reuse (ICIS-COMSAR'06). IEEE
7. Lind K, Heldal R (2009) Estimation of real-time software code size using COSMIC FSM. In: IEEE international symposium on object/component/ service-oriented real-time distributed computing, pp 244–248
8. Kaya M, Demirörs O (2011) E-Cosmic: a business process model based functional size estimation approach. In: 37th EUROMICRO conference on software engineering and advanced applications. IEEE, pp 404–408
9. Ferens DV. Software size estimation techniques, Air Force Institute of Technology (AFIT/ ISY), Wright-Patterson AEB, Ohio 45433, pp 701–706
10. Boehm BW (1986) Software engineering economics. Tutorial, software management: 3rd edn. IEEE Computer Society Press, Washington, DC, p 148
11. Boehm BW (1981) Software engineering economics. Prentice Hall, Englewood Cliffs, NJ
12. Zia Z, Rashid A, uzZaman K (2010) Software cost estimation for component based fourth-generation-language, IET Softw. 103–110
13. Rabbi MF, Natraj S, Kazeem OB (2009) Evaluation of convertibility issues between IFPUG and COSMIC function points. 4th International conference on software engineering advances. IEEE, pp 277–281